

# 第25-26课 PDF解析整理版

来源：飞书「ChatBI 项目系统蓝图」解析整理版。原课件 PDF 附件无下载权限，本文为文档正文导出。

## 第 25-26课 PDF 解析整理版

### 第25+26课：从报告生成到复杂案例实战

#### [要点] 课程总结与关系梳理

#### 核心关系

这两节课构成了ChatBI MVP第五阶段“多步执行与分析”的输出层构建与全链路实战验证的完整闭环：

- 第25课（输出层）：解决了“机器结果”到“业务报告”的转化问题。在多步执行链路跑通后，将分散、技术化的 `step_results` 整理为结构化的业务分析报告，补齐了面向业务人员的交付能力。
- 第26课（全链路实战）：以“利润下降原因分析”这一真实复杂问题为试金石，对包括第25课在内的所有模块进行端到端验证。重点暴露并修复了任务拆解过宽、维度越界、上下文传递丢失、SQL过长等工程化问题。

#### 学习路径总结

## 第 25 课：结果总结与自动分析报告生成

### 一、本节目标

#### 1.1 课程定位

第24课已经把多步执行链路跑通了，系统可以顺序执行多个子任务，也可以保存中间结果、记录步骤状态、处理失败和跳过。

#### 当前链路能返回

`step_results`，开发者能读懂，但业务人员读起来仍然费劲。业务人员通常需要的是一份整理过的分析报告：先看看到结论，再看到关键发现，最后知道下一步该查什么。

## 1.2 当前问题

沿着第 24 课的执行链路，系统已经可以查出下面两组结果（阶段性输出）：

1. 2026-05 的利润最低，为 920000 元
2. 动力电池-乘用车收入下降 8%，材料成本上升 5%

这些结果本身已经有用了，但它们还是分散的执行结果，还没有整理成一段完整的业务分析。

本节要处理的是输出层：把已经查出来的结果整理成一份业务人员可以直接阅读的分析报告。

## 1.3 本节课目标

1. 理解为什么多步执行结果还需要再做一次整理
2. 设计固定的报告结构：标题、执行摘要、关键发现、归因分析、趋势判断、行动建议
3. 实现 `report_generator.py`，把 `step_results` 转成结构化报告
4. 把报告生成能力接入 `PlanAndExecuteAgent.run()`
5. 用真实命令验证报告生成链路

## 二、核心思路

### 2.1 原始结果的局限性

第 24 课交付的 `step_results` 里已经有：`status`、`result_reference`、`rows`、`formatted`。

这些字段适合系统继续处理，但直接交给业务人员时，会遇到 3 个问题：

1. 结果是分散的：多个步骤各自返回一段结果，阅读时需要人工重新拼接；
2. 表达偏技术：`rows`、`sql`、`reference` 这些字段更像程序接口，不像分析结论；
3. 结论还没有收尾：系统已经查到了事实，但还没有把事实整理成一段完整说明。

业务人员更习惯看到下面这类输出：

- 关键发现是什么
- 利润下滑主要由哪几个因素驱动
- 这种趋势是短期波动，还是需要继续跟踪
- 接下来优先排查什么

### 2.2 输出层要满足的约束

报告生成不能只追求“写得像”，还要满足几个工程约束。

第一，结论必须基于已有执行结果来写。报告不能脱离自己补故事，更不能写出数据里没有出现的原因。

第二，结构必须稳定。只要报告字段顺序和层次能固定下来，便于后面做前端展示、报告存档和权限控制。

第三，链路要能回退。模型输出偶尔会偏离格式，系统需要在这种情况下继续返回一份可读报告，而不是整条链路直接断掉。

上述的 3 个工程约束，刚好对应本节课的 3 个目标：结构稳定、事实边界清楚、系统行为可控。

## 2.3 报告生成链路

本节课的核心处理链路：

链路过程拆解：

1. 先把多步骤结果压缩成适合总结的上下文
2. 再要求模型返回固定字段的 JSON
3. 接着用 Pydantic 校验字段和类型
4. 最后渲染成 Markdown；如果校验失败，则回退到模板化报告

报告结构收敛为 6 个字段：

先把报告结构固定下来，后续无论换模型、换前端、还是换展示方式，输出层都能保持稳定。

## 三、实现与演示

### 3.1 定义报告数据结构

第一步先定义 AnalysisReport，用 Pydantic 固定报告字段。

```
class AnalysisReport(BaseModel):
    title: str
    executive_summary: str
    key_findings: list[str] = Field(default_factory=list)
    root_causes: list[str] = Field(default_factory=list)
    trend_judgment: str
    action_suggestions: list[str] = Field(default_factory=list)
    markdown: str = ""
```

这一步的目标：

1. 明确报告边界
2. 给 JSON 校验提供固定入口

只要模型少返回字段，或者字段类型不对，系统就能立刻识别出来。

## 3.2 整理报告上下文

第二步先整理 `step_results`，把更适合总结的内容提出来。

```
def _compress_step_result(step_result: dict[str, Any]) -> dict[str, Any]:
    rows = step_result.get("rows") or []
    return {
        "step_id": step_result.get("step_id"),
        "step_name": step_result.get("step_name"),
        "status": step_result.get("status"),
        "success": step_result.get("success"),
        "formatted": step_result.get("formatted"),
        "result_reference": step_result.get("result_reference"),
        "rows_preview": rows[:3],
        "error": step_result.get("error"),
    }
```

这里保留 `formatted`、`rows_preview`、`status` 这些字段，是因为报告生成最需要先回答 3 件事：

1. 这一步查到了什么
2. 这一步是否成功
3. 这一步还有没有必要继续追踪

完整

SQL、完整结果集、内部实现细节，在这一层不需要全部带上。报告生成前先做一次上下文整理，把对结论有用的信号提出来。

## 3.3 模型返回结构化 JSON

第三步调用模型，期望的输出格式如下：

```
return (
    "请根据下面的 Agent 执行结果，生成一份结构化分析报告。\\n"
    "报告应包含：标题、执行摘要、关键发现、归因分析、趋势判断、行动建议。\\n"
    "4. 只返回 JSON，不要额外输出 Markdown。\\n"
    "5. JSON 必须包含 title、executive_summary、key_findings、"
    "root_causes、trend_judgment、action_suggestions 这 6 个字段。"
)
```

这样处理的目的：

1. 模型负责组织结论，代码负责校验结构，后续可以直接走 `json.loads()` 加 `AnalysisReport.model_validate()`
2. 先固定 JSON，再渲染 Markdown，因为 Markdown 更适合展示，JSON 更适合作为程序接口。

### 3.4 模板回退

模型输出并不总是可靠，所以这里还要准备一个回退方案。

```
try:
    raw_text = self.text_generator(system_msg, prompt)
    parsed = self._parse_report_json(raw_text)
    parsed.markdown = self._render_markdown(parsed)
    return parsed
except Exception:
    return self._build_fallback_report(...)
```

模板回退的目的：

1. JSON 格式错了，系统仍然能继续返回报告
2. 模型暂时不可用，系统仍然有基础输出
3. 后续做完整案例时，不会因为报告层的小波动把整条链路拖死

### 3.5 接回 Agent 主链路

最后把报告生成能力接回 PlanAndExecuteAgent.run()。

```
summary = self.summarizer.summarize(user_question, plan, step_results)
report = self.report_generator.generate(
    original_question=user_question,
    analysis_goal=plan.analysis_goal,
    step_results=[result.model_dump() for result in step_results],
    summary=summary.model_dump(),
)
```

更新后，Agent 的最终返回包含 4 类主要内容：plan、step\_results、summary、report。

这样做的目的是，开发者可以继续看执行明细，业务人员可以直接看分析报告。

### 3.6 示例验证

还是看这道问题：最近三个月利润为什么下降？

假设前两步已经执行成功，系统拿到了两组关键结果：

1. 2026-05 的利润最低，为 920000 元
2. 动力电池-乘用车收入下降 8%，材料成本上升 5%

这时需要做的工作，就是把这两组结果整理成一份可以直接阅读的报告。

本地实际运行输出如下：

```
# 最近三个月利润下降分析报告
```

**## 执行摘要**

利润下滑主要来自动力电池-乘用车业务收入回落，同时材料成本继续上升。

**## 关键发现**

- 2026-05 的利润最低，为 920000 元。
- 动力电池-乘用车收入下降 8%，材料成本上升 5%。

**## 归因分析**

- 核心产品线收入回落，直接拖低毛利。
- 材料成本抬升进一步压缩利润空间。

**## 趋势判断**

如果订单恢复速度不足，短期利润压力仍会延续。

**## 行动建议**

- 优先复盘动力电池-乘用车订单流失原因。
- 单独跟踪材料采购价格与毛利率变化。

这份报告和原始 `step_results` 相比，有 3 个明显变化：

1. 原始结果是分步返回，报告已经按业务逻辑重新整理
2. 原始结果偏数据事实，报告补上了归因和趋势表达
3. 原始结果适合程序消费，报告已经适合业务阅读

### 3.7 代码验证

#### 验证一：报告生成器

1. 模型返回合法 JSON 时，系统能正确解析并渲染 Markdown
2. 模型返回非法内容时，系统能回退到模板化报告

```
$ uv run pytest tests/test_report_generator.py -vv
===== test session starts =====
platform darwin -- Python 3.12.9, pytest-9.1.1, pluggy-1.6.0
collecting ... collected 2 items

tests/test_report_generator.py::test_report_generator_pares_structured_llm_output PASSED [ 50%]
tests/test_report_generator.py::test_report_generator_falls_back_to_template_when_llm_output_is_invalid PASSED [100%]

===== 2 passed in 0.31s =====
```

#### 验证二：Agent 集成后的完整行为

1. 原来的执行链路没有被破坏
2. `PlanAndExecuteAgent.run()` 已经能稳定返回 `report`

```
$ uv run pytest tests/test_agent_planner.py -vv
===== test session starts =====
platform darwin -- Python 3.12.9, pytest-9.1.1, pluggy-1.6.0
collecting ... collected 9 items
...
tests/test_agent_planner.py::test_agent_returns_business_report_after_execution PASSED [ 44%]
```

```
...
===== 9 passed in 19.83s =====
```

## 四、小结

### 4.1 本节收获

这一节完成的是 Agent 的输出层，当前链路已经可以把多步 SQL 执行结果整理成一份结构化分析报告。

本节内容总结：

1. 定义结构化报告模型
2. 把 `step_results` 压缩成适合总结的上下文
3. 用 JSON + Pydantic 约束模型输出格式
4. 在格式异常时回退到模板化报告

### 4.2 常见问题

#### 4.2.1 能否让模型直接输出整篇 Markdown

可以这样做，但校验会更麻烦。Markdown 更适合普通展示，不适合作为程序接口；先收成 JSON，再统一渲染 Markdown（支持后期按照固定模板渲染），会更稳定。

#### 4.2.2 为什么还要保留 summary

summary 和 report 解决的问题不同：

- summary 偏执行状态汇总
- report 偏业务分析表达

前者更适合系统和开发者，后者更适合业务人员。

#### 4.2.3 如果步骤失败了，报告还能不能生成

可以，但结论需要保守。报告应明确说明“当前只覆盖已成功步骤”，不能把未完成的部分写成已经确认的结论。

报告生成模块既要能整理结论，也要保留事实边界，还要在异常场景下继续给出可读输出。

### 4.3 本节主线回看

完成这一节后，第五阶段已经具备了：

1. 拆解问题
2. 规划执行
3. 多步查询
4. 中间结果管理
5. 自动生成分析报告

下一节会把这些能力一次性串起来，完成“利润下降原因分析系统”的完整复杂案例。

## 五、附录

### 5.1 本节代码路径

- code/chatbi\_mvp/report\_generator.py
  - code/chatbi\_mvp/agent\_planner.py
  - code/chatbi\_mvp/llm\_client.py
  - code/chatbi\_mvp/tests/test\_report\_generator.py
  - code/chatbi\_mvp/tests/test\_agent\_planner.py
- 

## 第 26 课：复杂案例完整实战：利润下降原因分析系统

### 一、本节目标

#### 1.1 课程定位

回顾第 21-25 课：

- 第 22 课完成任务拆解
- 第 23 课完成 Planner 和 Executor 骨架
- 第 24 课补齐中间结果管理
- 第 25 课补齐报告生成

本节课定位：用一个真实复杂问题把第五阶段完整从头跑到尾，发现链路中的重点、易错点并思考解决方案【meta 方案：关于技术方案的修复方案】。

## 1.2 当前问题

围绕“最近三个月利润为什么下降”这类问题，前面我们已经证明过：单条 SQL 无法解决该问题。

思考：将复杂问题拆解需要的各个模块直接串联起来，是否就已经可以解决问题了？

真实跑起来之后，链路里一定会暴露各种问题（重点考察）：

1. 任务拆解过宽，子任务数量很容易膨胀
2. 拆解阶段会出现数据库里并不存在的维度
3. 前置步骤结果传给下游时有信息损失
4. step 过宽之后，SQL 变长，系统稳定性下降

## 1.3 本节课目标

1. 看清完整链路初次运行时暴露的问题
2. 找到这些问题对应的根因
3. 修复思路用代码实现
4. 真实运行结果验证

## 二、典型问题分析

### 2.1 任务拆解过宽

#### 2.1.1 问题表现

先看问题：分析近半年的利润变化情况

```
uv run python agent_planner.py "分析近半年的利润变化情况"
```

初始版本的任务拆解，被拆成了 15 个步骤，前 8 个步骤如下：

```
plan_steps: 15
1. 统计近半年月度利润趋势
2. 统计近半年月度收入趋势
3. 统计近半年月度销售成本趋势
4. 统计近半年月度期间费用趋势
5. 统计近半年月度毛利趋势
6. 统计近半年月度毛利率趋势
7. 统计近半年月度利润环比变化
8. 按产品线分析近半年利润贡献
.....
```

#### 2.1.2 根因思考

这里的问题核心在于：拆解器拿到的边界不够完整。

当拆解器只拿到自然语言问题时，它会倾向于按语义习惯把问题补得很全：趋势要拆、环比要拆、维度要拆、原因也要拆。

为了让问题拆解结果更加聚焦、精准，我们应该明确告诉它：

- 1. 当前数据库里有哪些维度
- 2. 当前系统里有哪些指标
- 3. 这一类问题最多应该拆成几步

2.1.3 解决思路

为了解决上述问题，我们针对问题拆解步骤，可以尝试 3 个优化：

- 1. 在拆解 Prompt 里注入 SCHEMA
- 2. 显式注入可用分析维度和可用指标
- 3. 按问题类型限制任务数，超出后重试一次

这类问题的目标，是把问题拆成当前系统能执行、每一步都比较简单的子任务。

2.1.4 验证结果

修复后，还是同一个问题“分析近半年的利润变化情况”，真实运行中的计划收敛成 6 步：

```
plan_steps: 6
1. 近半年月度利润趋势
2. 近半年月度收入趋势
3. 近半年月度销售成本趋势
4. 近半年月度期间费用趋势
5. 近半年月度毛利率趋势
6. 异常月份利润变化原因归纳
```

任务数量收住之后，后续步骤的执行压力明显下降。

2.2 维度超出数据模型

2.2.1 问题表现

完整运行里，最直接的一类错误就是引用了数据库里不存在的字段。

下面的结果是一次真实的失败输出（o.channel 并不存在于当前数据模型里）：

```
failed_step: 分析业务维度对利润下降的贡献
failed_error: (1054, "Unknown column 'o.channel' in 'field list'")
failed_sql_head:
```

```
WITH monthly_order_metrics AS (  
    SELECT DATE_FORMAT(o.order_date, '%Y-%m') AS month,  
    ...  
    o.region, o.channel,  
    ...  
)
```

2.2.2 根因思考

这个问题在任务拆解阶段就已经埋下了，到了 SQL 生成阶段才被真正暴露出来。

只要上游把 channel 当成一个合法维度传下去，后面的 Planner 和 Text2SQL 就会继续沿着这个方向往下生成。

因此，这里的问题本质上是：拆解阶段没有把可用维度边界限定住。

2.2.3 解决思路

修复方法仍然是集中在拆解器：

- 1. 先给模型明确的维度清单
  - 2. 对拆解结果里的 dimensions 做合法性校验
  - 3. 如果用户问题里提到未建模维度，回退到最接近的已建模维度
- 这一步的重点，是让拆解器从源头上只输出当前系统能承接的维度。

该类问题优化的典型错误思路：遇到实际错误，再逐个补充“禁止 channel”这种规则（只适合临时兜底，不适合作为正式方案）。

2.2.4 验证结果

修复后，同一类运行记录里，维度步骤已经不再出现渠道，计划项收敛为：

- 1. 查看最近三个月利润趋势
- 2. 拆解收入、成本、费用的变化
- 3. 定位利润下滑最明显的业务维度
- 4. 开展费用项目专项分析
- 5. 汇总利润下降主要原因并排序

对应的最终状态也会从失败变成：

```
completed: 5  
failed: 0  
skipped: 0
```

2.3 前置结果传递丢信息

### 2.3.1 问题表现

早期运行里，下游步骤拿到的前置结果是这种形式：

context used head:

- 确认最近三个月利润趋势及下降幅度:

或者：

context used head:

- 统计近半年各月利润趋势:

可以看到，下游拿到的是被压缩、排版过的文本，而且里面还夹着表格边框。

### 2.3.2 根因思考

前置结果有两种不同的下游用途，需要的数据形态并不一样：

1. 给人看
2. 给下一个步骤继续执行

如果直接把摘要文本或格式化表格传给下游，会有两个后果：

1. 细节丢失
2. 展示层噪声混进执行链路

### 2.3.3 解决思路

这部分修复落在 `agent planner.py`。

```
StepExecutor._build_dependency_context()
```

后面改为传结构化

JSON, 至少包含: step id、step name、columns、rows、result reference。

摘要保留给总结和报告使用，执行链路里直接传结构化结果。

### 2.3.4 验证结果

修复后，同样是前置上下文，真实输出变成：

```
context_used_head: {"step_id": "step_1", "step_name": "近半年月度利润趋势", "columns": ["month",
"revenue_cny", "sales_cost", "period_expense", "profit"], "rows": [{"month":
"2026-01-01", "revenue_cny": "18802524.335000", "sales_cost": "1453880.000...
```

上下文从“表格文本”变成“结构化结果”，下游步骤可用的信息明显更多，也更干净。

## 2.4 SQL 过长导致执行失败

### 2.4.1 问题表现

当 step 过宽时，SQL 会变得很长，失败形式也会比较多样。

如维度步骤导致的 SQL 语法错误：

```
failed_error: (1064, "You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 141")
```

如 UNION 不合法：

```
failed_step: 定位利润下滑最明显的业务维度  
failed_error: (1271, "Illegal mix of collations for operation 'UNION'")
```

### 2.4.2 根因思考

错误的问题根因：“模型输出长度不够”。

实际上更主要的原因：

1. 拆解和计划不合理，导致单步职责太宽
2. 维度分析被塞进一条很重的 SQL，里面同时做了太多拼接和比较

token 上限会影响结果，但根因还在前面的拆解和计划。

### 2.4.3 解决思路

基于根因分析的结论，我们的处理分成两步：

1. 先把任务和 step 收窄，让每一步尽量简单；
2. 再把 LLM\_MAX\_TOKENS 提高到 4000 作为兜底，减少长 SQL 被截断的概率。

对应代码位置：query\_decomposer.py、agent\_planner.py、config.py

### 2.4.4 验证结果

修复后的真实运行记录里，完整状态已经变成：

```
completed: 6  
failed: 0  
skipped: 0
```

说明“先收窄 step，再做 token 兜底”这条思路是有效的。

## 三、修改合并与完整运行

### 3.1 修改内容总结

这一轮代码修改主要集中在 4 个文件里。

#### 3.1.1 query\_decomposer.py

目标：优化任务拆解。

修改点：

1. 注入 SCHEMA
2. 注入可用维度
3. 注入可用指标
4. 对维度合法性做校验
5. 对任务数量做限制并在必要时重试

#### 3.1.2 agent\_planner.py

这一部分负责把步骤间的数据契约改对。

修改点：

1. step question 收回到较简单的执行约束
2. 下游不再依赖摘要文本
3. 前置结果改为结构化 JSON 传递

#### 3.1.3 prompt\_builder.py

强化 SQL 生成的要求：

1. SQL 语法正确性约束
2. 不允许引用 Schema 中不存在的字段
3. 困难问题类别 few shot

#### 3.1.4 config.py

这一部分负责 SQL 生成的兜底：

1. LLM\_MAX\_TOKENS 默认值提高到 4000

### 3.2 完整运行

命令如下：

```
cd code/chatbi_mvp
uv run pytest -q
uv run python agent_planner.py "分析近半年的利润变化情况"
```

修复后的完整运行记录里，计划和执行状态如下：

```
plan_steps: 6
1. 近半年月度利润趋势
2. 近半年月度收入趋势
3. 近半年月度销售成本趋势
4. 近半年月度期间费用趋势
5. 近半年月度毛利率趋势
6. 异常月份利润变化原因归纳

completed: 6
failed: 0
skipped: 0
```

#### 3.2.1 运行结果

利润趋势：

| month      | revenue_cny     | sales_cost   | period_expense | profit          |
|------------|-----------------|--------------|----------------|-----------------|
| 2026-01-01 | 18802524.335000 | 1453880.0000 | 3166442.00     | 14182202.335000 |
| 2026-02-01 | 2033200.000000  | 110000.0000  | 4991435.00     | -3068235.000000 |

收入趋势：

| month      | revenue_cny     |
|------------|-----------------|
| 2026-01-01 | 18802524.335000 |
| 2026-02-01 | 2033200.000000  |

销售成本趋势：

| month      | sales_cost   |
|------------|--------------|
| 2026-01-01 | 1841785.0000 |
| 2026-02-01 | 523620.0000  |

#### 3.2.2 最终效果

最终报告摘要里的真实结论如下：

从已提供的月度结果看，近半年利润波动较大。2026-01 利润为 14182202.335；

2026-02 利润转为 -3068235.000; 2026-05 利润为 -244863.150, 较 2 月有所修复但仍未恢复为正。

现有数据表明, 利润波动的核心影响因素主要是收入规模变化, 其次是期间费用上升。

## 四、小结

本节课重点:

1. 完整执行第五阶段任务
2. 定位、分析并解决链路中重点问题

附件(飞书无下载权限): 第 25 课: 结果总结与自动分析报告生成.pdf

附件(飞书无下载权限): 第 26 课: 复杂案例完整实战: 利润下降原因分析系统.pdf